# Compare Search Trees

Author: YiFeng Zheng

YuJie Sun

Group number: 17

Date: 2026-10-03

**Abstract**

This report investigates the performance of five search trees on sequences of insertions and deletions: unbalanced binary search trees, AVL trees, splay trees, red-black trees, and B+ trees. Correctness is checked by an insertion/deletion smoke test for each tree, run under memory and undefined-behavior sanitizers. Each timed configuration is repeated three times. Ordered BST insertion exhibits quadratic growth, while balanced and adaptive trees avoid that total-cost behavior. Phase-separated plots reveal the benefits of splaying on ordered sequences. All programs are implemented in C.

# Contents

**1 Introduction** **3**
1.1 Background and Objectives . . . 3
1.2 Problem Statement . . . 3

**2 Data Structure / Algorithm Specification** **3**
2.1 Unbalanced Binary Search Tree . . . 3
2.2 AVL Tree . . . 3
2.3 Splay Tree . . . 4
2.4 Red-Black Tree . . . 4
2.5 B+ Tree . . . 5
2.6 Correctness Argument . . . 6

**3 Testing Results** **6**
3.1 Environment and Timing Methodology . . . 6
3.2 Smoke Test . . . 7
3.3 Runtime Tables . . . 8
3.3.1 Increasing insertion / increasing deletion . . . 8
3.3.2 Increasing insertion / reverse deletion . . . 8
3.3.3 Random insertion / random deletion . . . 9
3.4 Runtime Plots . . . 9

**4 Analysis and Comments** **11**
4.1 Time Complexity . . . 11
4.1.1 Why Sorted BST Updates Are Quadratic . . . 11
4.1.2 Why Splay Benefits from Ordered Updates . . . 11
4.2 Space Complexity . . . 12
4.3 Measured Comparison . . . 12
4.4 Limitations and Improvements . . . 13
4.5 Conclusion . . . 13

**5 References** **14**

**6 Author List** **14**
6.1 Division of Work . . . 14

**7 Declaration** **14**

**8 Signatures** **14**

**9 Source Code Appendix** **14**

# 1 Introduction

## 1.1 Background and Objectives

A search tree represents an ordered set while supporting insertion, deletion, and membership queries. The same set can have very different update costs because its representation depends on balance rules and access order.

This project compares an unbalanced binary search tree (BST), an AVL tree, a splay tree, a red-black tree, and a B+ tree using the three update sequences.

## 1.2 Problem Statement

For each size N, the keys are the N distinct integers from 0 through N-1. The experiment inserts every key into an empty tree and then deletes every key.

1. Scenario 1 inserts in increasing order and deletes in increasing order;
2. Scenario 2 inserts in increasing order and deletes in decreasing order;
3. Scenario 3 uses separately shuffled insertion and deletion permutations. The measured sizes are 1,000, 2,000, 5,000, 10,000, 20,000, 50,000, and 100,000.

# 2 Data Structure / Algorithm Specification

## 2.1 Unbalanced Binary Search Tree

As introduced in FDS, a binary search tree is a rooted binary tree in which each node's key is greater than every key in its left subtree and less than every key in its right subtree [1].

We use an unbalanced BST as the baseline to highlight the balancing properties of the other trees.

Our C implementation inserts and deletes iteratively. Insertion attaches a new leaf; deletion bypasses a node with at most one child or replaces a two-child node's key with its inorder successor [1]. Updates perform no balancing rotations.

## 2.2 AVL Tree

An AVL tree is a BST in which every node's left and right subtree heights differ by at most one [2].

Compared with an unbalanced BST, it imposes a strict height-balance constraint.

`AvlBalance` repairs ancestors after recursive updates. It uses single rotations for LL/RR and double rotations for LR/RL imbalances, reconnecting child pointers and refreshing cached heights [2].

Insertion follows the BST search path and creates a leaf. Deletion bypasses a node with at most one child; for two children, it copies the inorder successor's key and recursively deletes that successor. Both operations call `AvlBalance` on the return path and reconnect the returned subtree root.

Let $b(T) = h(T_L) - h(T_R)$, with empty-subtree height -1. If $b(T) > 1$, a nonnegative left-child balance calls for a right rotation; otherwise, rotate the left child left before rotating T right. If $b(T) < -1$, a nonpositive right-child balance calls for a left rotation; otherwise, rotate the right child right before rotating T left. A child balance of zero therefore uses a single rotation, which matters after deletion. Each rotation updates the demoted node's height before the promoted node's height. Repairs continue up the recursive return path, since deletion may shorten several ancestor subtrees.

### 2.3 Splay Tree

A splay tree is a self-adjusting BST that moves an accessed node to the root through rotations [3].

Its distinguishing feature is access-driven restructuring, without a fixed balance constraint.

The C code uses top-down splaying: zig-zig rotations shorten repeated-direction paths, while smaller and larger keys are collected separately and reassembled around the accessed root [3]. Insertion makes the new key the root; deletion joins the subtrees by splaying the left subtree's maximum. Membership checks are read-only and do not splay.

### 2.4 Red-Black Tree

A red-black tree is a BST with red or black nodes, a black root, no adjacent red nodes, and equal numbers of black nodes on all paths from a given node to its descendant null leaves [1].

Beyond the BST ordering rule, it uses color metadata and these constraints to maintain balance.

Our implementation uses the left-leaning variant described in [1]. Rotations promote a child, transfer the former root's color to it, and color the demoted node red; `FlipColors` toggles the parent and both children. Deletion uses `MoveRedLeft` or `MoveRedRight` before descending into a 2-node, and the final root is black.

Insertion follows the BST path, creates a red leaf, and calls `RedBlackBalance` on return. This function applies three checks in order: rotate left if the right link is red; rotate right if the left and left-left links are red; flip colors if both child links are red. The public insertion function then colors the root black.

Deletion first checks membership and returns unchanged if the key is absent. If both root children are black, it temporarily colors the root red. Null links count as black. The recursive deletion uses these branches:

1. Before descending left, if the left and left-left links are both black, call `MoveRedLeft`. It flips the parent and child colors. If the right-left link is red, it then rotates the right child right, rotates the current node left, and flips colors again.
2. In the other branch, first rotate right if the left link is red. If the current key matches and there is no right child, remove the node. Otherwise, if the right and right-left links are both black, call `MoveRedRight`. It flips colors and, when the left-left link is red, rotates right and flips colors again.
3. After these adjustments, a matching key is replaced by its inorder successor, which is removed by `DeleteMin`; otherwise, deletion continues right. `DeleteMin` uses the same left-branch preparation. Each return calls `RedBlackBalance`, and the public function restores a black root if the tree is nonempty.

The color moves prepare a branch to lose a key without leaving an unmatched black-height decrease. The return-path checks restore the left-leaning form.

### 2.5 B+ Tree

A B+ tree is a balanced multiway search tree with separator keys in internal nodes and records in linked leaves at the same depth [4].

Unlike the binary branching of a BST, it allows multiple children per node, enforces occupancy constraints, and stores records separately in leaves.

Following the split and redistribution rules in [4], the C code splits an overflowing node into 16 and 17 entries, propagating splits upward. Deletion borrows from a sibling with spare entries or merges adjacent nodes; a one-child root is replaced by its child. `Refresh` updates separator keys, and leaf splits and merges maintain the `Next` links.

Here, an entry means a record in a leaf or a child pointer in an internal node. Each non-root node holds 16 to 32 entries. Internal separator keys are the minimum keys of all children except the first; a key equal to a separator goes to the child on its right.

Insertion finds the target leaf and shifts records to make room. If its count reaches 33, the left node keeps 16 entries and a new right node gets 17. The parent inserts the new child pointer and refreshes its separators. An internal overflow is split in the same way; a root split creates a new root with two children.

Deletion removes the record and shifts later records left. On return, `RepairChild` leaves a child with at least 16 entries unchanged. Otherwise, it first tries to borrow the left sibling's last entry, then the right sibling's first entry; the donor must have more than 16 entries. If neither can lend, it merges with the left sibling when available, or with the right sibling otherwise. The parent loses one child pointer, so underflow may continue upward. Leaves move records and repair `Next` after

merging; internal nodes move child pointers. Affected minima and separators are refreshed. A one-child internal root is removed, and an empty leaf root becomes NULL.

### 2.6 Correctness Argument

An empty tree is correct. Assume the tree is correct before an update. BST deletion uses the right subtree's smallest key when replacing a node with two children, so keys stay in order [1]. Rotations also keep this order; AVL and red-black repairs restore their balance rules [1], [2]. Splay deletion joins the left and right trees only when all left keys are smaller [3]. B+ splits, borrowing, and merges keep records in order and within node limits; updated separator keys keep searches correct [4]. These steps add or remove only the requested key, so each update leaves a correct tree. By induction, this holds for the whole sequence. Tests check some cases but do not prove all cases.

## 3 Testing Results

### 3.1 Environment and Timing Methodology

The experiment was conducted in the following environment.

| Item | Configuration |
|---|---|
| Experiment date | 2026-10-03 |
| CPU | AMD Ryzen 7 H 255 W/ Radeon 780M |
| Operating system | WSL2; Linux 6.6.87.2-microsoft-standard-WSL2; x86-64 |
| Compiler | gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 |
| Build flags | -std=c11 -O2 -Wall -Wextra -Wpedantic |
| Timer | POSIX CLOCK_MONOTONIC, elapsed wall time |
| Repetitions | 3 for each scenario / size / tree |
| Execution | One process; sequential; no CPU affinity |
| B+ capacity | 32 leaf records or 32 internal children |

### 3.2 Smoke Test

Observed smoke test results.

| Test case | Type | Expected / actual result |
|---|---|---|
| Deleting from an empty tree | Boundary | Accepted, no change; PASS for all five trees |
| INT_MIN and INT_MAX; 1,000 increasing keys | Functional | Both boundary keys found; sampled keys found; an absent key reported absent |
| Full insert-then-delete cycle | Functional | Every key accepted, then removed; NULL root afterwards; PASS |
| Same cycle under ASan and UBSan | Memory / undefined behavior | No sanitizer diagnostics; PASS |
| All 315 timed trials, N up to 100,000 | Large-scale | Sampled keys present after insertion; NULL root after deletion; PASS |

README.md documents how to reproduce every result in this chapter. The smoke test checks sampled membership and a full deletion cycle; it does not replace those structural checks.

## 3.3 Runtime Tables

- Times are in milliseconds (ms); N is the number of keys.
- Each value is the median total insertion-plus-deletion time of three runs.

### 3.3.1 Increasing insertion / increasing deletion

Increasing insertion / increasing deletion: median total elapsed time (ms).

| N | BST | AVL | Splay | RedBlack | BPlus32 |
|---|---|---|---|---|---|
| 1,000 | 1.151 | 0.055 | 0.027 | 0.143 | 0.054 |
| 2,000 | 4.646 | 0.202 | 0.084 | 0.392 | 0.186 |
| 5,000 | 27.751 | 0.408 | 0.176 | 0.876 | 0.394 |
| 10,000 | 111.839 | 1.163 | 0.447 | 2.279 | 1.012 |
| 20,000 | 444.720 | 1.644 | 0.788 | 4.374 | 1.797 |
| 50,000 | 2757.504 | 7.110 | 1.843 | 14.321 | 5.003 |
| 100,000 | 10999.209 | 16.004 | 3.402 | 37.267 | 11.107 |

### 3.3.2 Increasing insertion / reverse deletion

Increasing insertion / reverse deletion: median total elapsed time (ms).

| N | BST | AVL | Splay | RedBlack | BPlus32 |
|---|---|---|---|---|---|
| 1,000 | 2.348 | 0.087 | 0.036 | 0.179 | 0.100 |
| 2,000 | 9.652 | 0.173 | 0.097 | 0.321 | 0.209 |
| 5,000 | 57.488 | 0.422 | 0.195 | 0.839 | 0.592 |
| 10,000 | 219.320 | 0.708 | 0.381 | 1.763 | 0.986 |
| 20,000 | 886.977 | 1.621 | 0.779 | 4.072 | 2.177 |
| 50,000 | 5483.877 | 5.159 | 1.950 | 13.683 | 6.083 |
| 100,000 | 21933.532 | 15.375 | 3.894 | 36.391 | 14.964 |

### 3.3.3 Random insertion / random deletion

Random insertion / random deletion: median total elapsed time (ms).

| N | BST | AVL | Splay | RedBlack | BPlus32 |
|---|---|---|---|---|---|
| 1,000 | 0.077 | 0.126 | 0.118 | 0.201 | 0.121 |
| 2,000 | 0.172 | 0.298 | 0.296 | 0.430 | 0.241 |
| 5,000 | 0.555 | 0.881 | 0.877 | 1.233 | 0.777 |
| 10,000 | 1.283 | 2.471 | 2.521 | 3.278 | 2.421 |
| 20,000 | 2.661 | 4.235 | 4.836 | 6.370 | 3.356 |
| 50,000 | 9.664 | 15.876 | 15.423 | 19.003 | 10.389 |
| 100,000 | 22.791 | 37.573 | 41.841 | 52.854 | 22.443 |

## 3.4 Runtime Plots

- Panels show insertion, deletion, and total time.
- Axes show N and seconds, both on logarithmic scales.
- Colors and markers identify trees; points show three-run medians and error bars show the observed range.

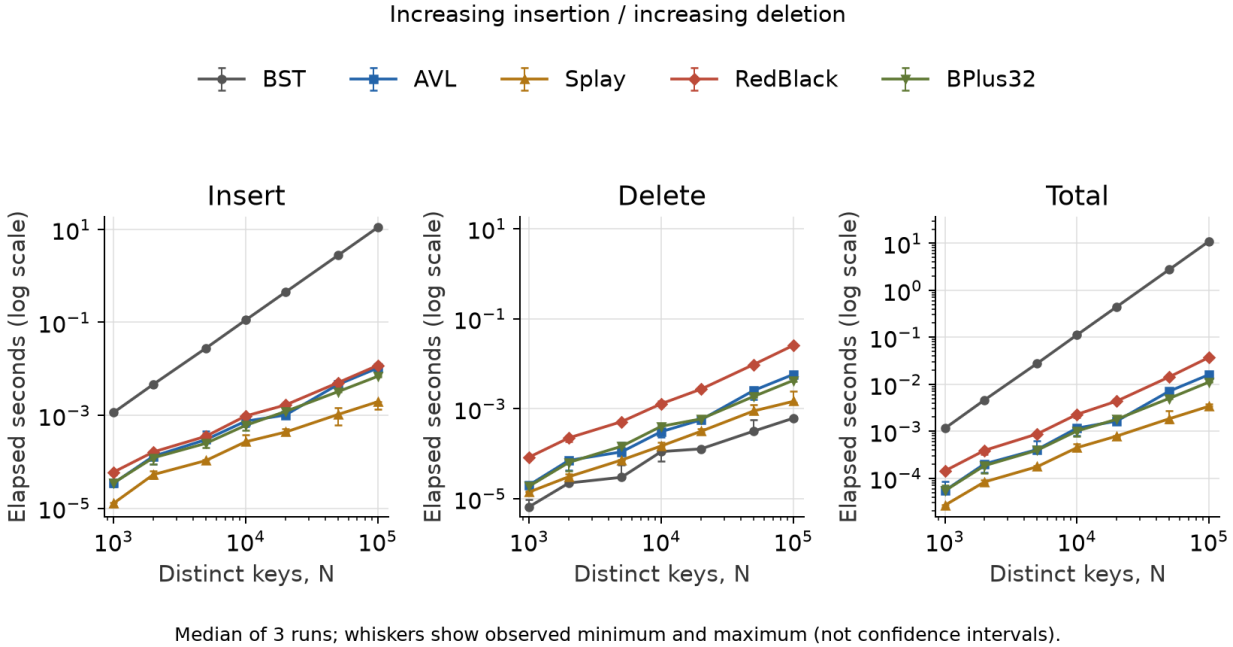



Figure 1: Increasing insertion / increasing deletion. Each phase uses the same scale as its counterpart in the other scenarios.

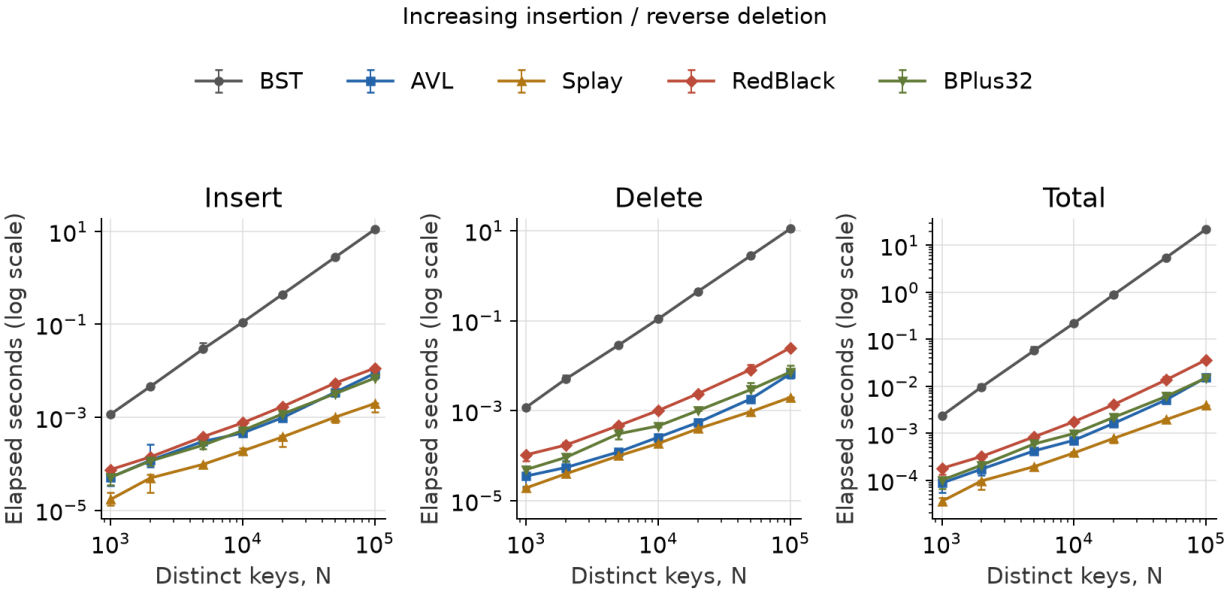



Figure 2: Increasing insertion / reverse deletion. Each phase uses the same scale as its counterpart in the other scenarios.

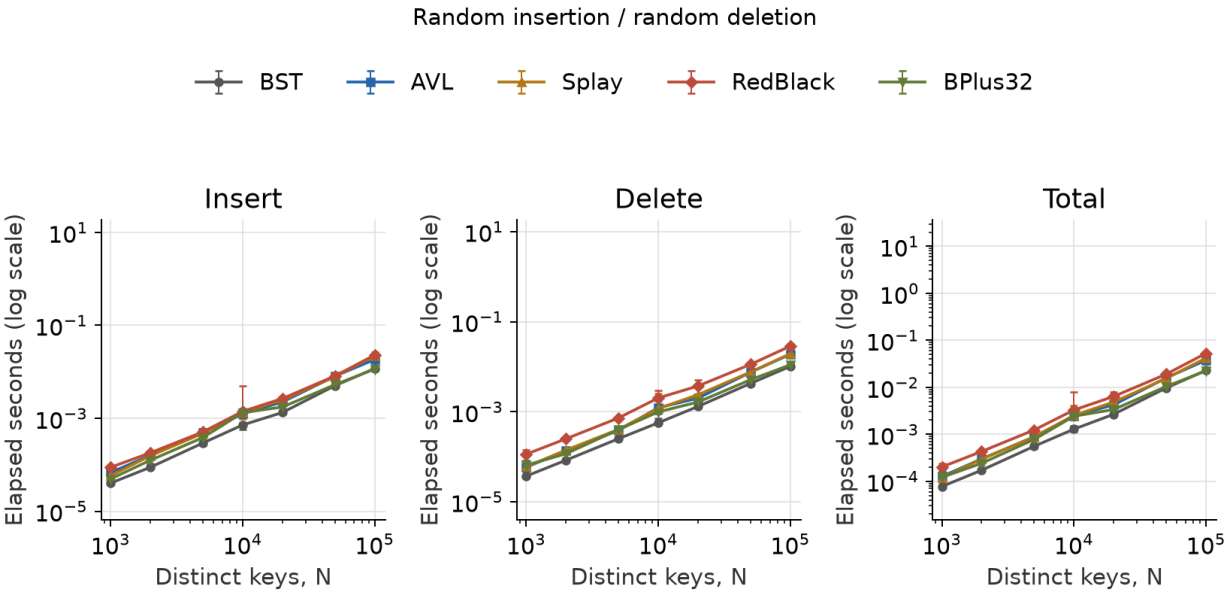



Figure 3: Random insertion / random deletion. Each phase uses the same scale as its counterpart in the other scenarios.

# 4 Analysis and Comments

## 4.1 Time Complexity

The full sequence consists of N insertions followed by N deletions. Bounds are worst-case unless marked as amortized; the B+ tree capacity is fixed at 32.

BST updates take time proportional to tree height [1]. AVL and red-black balancing maintain logarithmic height [1], [2]; fixed-capacity B+ trees also have logarithmic height [4]. Splay operations have an $O(\log N)$ amortized bound [3]. Applying these bounds to the 2N updates gives the full-sequence bounds below.

| Tree | Single insertion or deletion | Full sequence |
|---|---|---|
| BST | $O(N)$ | $O(N^2)$ |
| AVL | $O(\log N)$ | $O(N \log N)$ |
| Splay | $O(N)$; $O(\log N)$ amortized | $O(N \log N)$ |
| Red-Black | $O(\log N)$ | $O(N \log N)$ |
| B+ (capacity 32) | $O(\log N)$ | $O(N \log N)$ |

### 4.1.1 Why Sorted BST Updates Are Quadratic

Increasing insertion creates a right chain in the plain BST. Using the height-based update bound [1], the ith insertion visits i−1 nodes, giving N(N−1)/2 visits and $\Theta(N^2)$ time.

At the same time, increasing deletion removes the root each time and takes $\Theta(N)$ in total. Reverse deletion searches the remaining chain each time and takes $\Theta(N^2)$. Thus, both scenarios have quadratic total cost, but different deletion costs.

### 4.1.2 Why Splay Benefits from Ordered Updates

With increasing insertions, the largest key stays at the root, so each insertion takes constant time. The tree may become a left chain, but the total insertion cost is still $O(N)$.

Deleting in reverse order also takes $O(N)$: each deletion removes the maximum at the root and exposes its left subtree.

Increasing deletions behave differently. The first deletion must traverse the left chain to reach the minimum, costing $\Theta(N)$, so not every deletion is constant-time. However, the general amortized theorem gives $O(N \log N)$ for the full update sequence [3].

### 4.2 Space Complexity

| Tree | Total tree space | Update auxiliary space |
|---|---|---|
| BST | $O(N)$ | $O(1)$ |
| AVL | $O(N)$ | $O(\log N)$ |
| Splay | $O(N)$ | $O(1)$ |
| Red-Black | $O(N)$ | $O(\log N)$ |
| B+ (capacity 32) | $O(N)$ | $O(\log N)$ |

All five trees use linear storage. Auxiliary space is constant for iterative updates and logarithmic for the recursive update stacks in these implementations.

### 4.3 Measured Comparison

1. Increasing insertion / increasing deletion: At N = 100,000, Splay is fastest (3.402 ms), while BST is slowest (10,999.209 ms), a 3,232.85-fold difference.
2. Increasing insertion / reverse deletion: Splay is fastest (3.894 ms), while BST is slowest (21,933.532 ms), a 5,633.15-fold difference.
3. Random insertion / random deletion: BPlus32 is fastest (22.443 ms), while RedBlack is slowest (52.854 ms), a 2.35-fold difference.
4. Performance rankings: From fastest to slowest, both ordered workloads rank Splay, BPlus32, AVL, RedBlack, BST; the random workload ranks BPlus32, BST, AVL, Splay, RedBlack.
5. Scope: These rankings describe the results at N = 100,000 in this experiment and do not establish a universal ranking.

The small gap in the random workload needs care: BPlus32 takes 22.443 ms and BST takes 22.791 ms at N = 100,000, a difference of about 1.5% relative to BST. With only three repetitions, these results show close performance rather than a clear advantage. BST has the lower median at every smaller tested size.

Random insertion does not force the one-sided chain produced by sorted keys. Our BST also uses iterative updates without height updates, color changes, or balancing rotations. This lower maintenance cost is a plausible reason for its good random-workload times, even when its search paths are longer than those of a balanced tree. It does not guarantee short paths for every random trial or throughout deletion.

AVL and red-black trees have the same asymptotic update bound, but their implementations do different work. AVL updates cached heights and rotates when needed. Our left-leaning red-black deletion first runs `RedBlackFind`, then traverses from the root again to delete the key. It also adjusts colors and links before descent and checks balance on return. These extra steps help explain why RedBlack is slower here; they do not establish that red-black trees are generally slower than AVL trees.

BPlus32 trades work inside nodes for fewer tree levels. Internal routing uses binary search, and storing several keys together may improve memory locality. However, leaf updates scan and shift entries, while `Refresh` rebuilds separators along the return path. Splits, borrowing, and merges add more array work. This tradeoff is consistent with BPlus32's competitive times and its varying advantage over AVL across sizes and workloads. The test performs no disk I/O or leaf-chain range scans, so neither explains the measured speed.

These explanations follow the code structure. The experiment does not count node visits, rotations, color changes, or cache misses, so it cannot isolate their contributions to elapsed time.

### 4.4 Limitations and Improvements

- Statistics: only three runs; use more repetitions to assess small differences.
- Workloads: only build-then-delete integer tests; add mixed operations and search benchmarks.
- Page I/O: this in-memory benchmark does not measure disk access; test a disk-backed B+ tree.

### 4.5 Conclusion

The results show that operation order affects tree performance.

Sorted insertions turn a plain BST into a chain, giving quadratic total time. AVL and red-black trees maintain logarithmic height through balancing.

In this experiment, Splay was fastest for both ordered workloads, while BPlus32 was fastest for the random workload at N = 100,000. These results show that the best choice depends on the workload, within the implementation and measurement limits discussed above.

# 5 References

[1] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011, secs. 3.2-3.3.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2014, sec. 4.4.

[3] D. D. Sleator and R. E. Tarjan, "Self-adjusting binary search trees," *J. ACM*, vol. 32, no. 3, pp. 652-686, Jul. 1985, doi: 10.1145/3828.3835.

[4] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019, ch. 14.

# 6 Author List

## 6.1 Division of Work

YiFeng Zheng: Implemented the programs, conducted the tests, and wrote the report.

YuJie Sun: Reproduced the tests and reviewed the report.

# 7 Declaration

We hereby declare that all work in this project, titled "Compare Search Trees," is our independent effort as a group.

# 8 Signatures

YiFeng Zheng

YuJie Sun

# 9 Source Code Appendix

The full source code is provided in the accompanying `code/` directory.